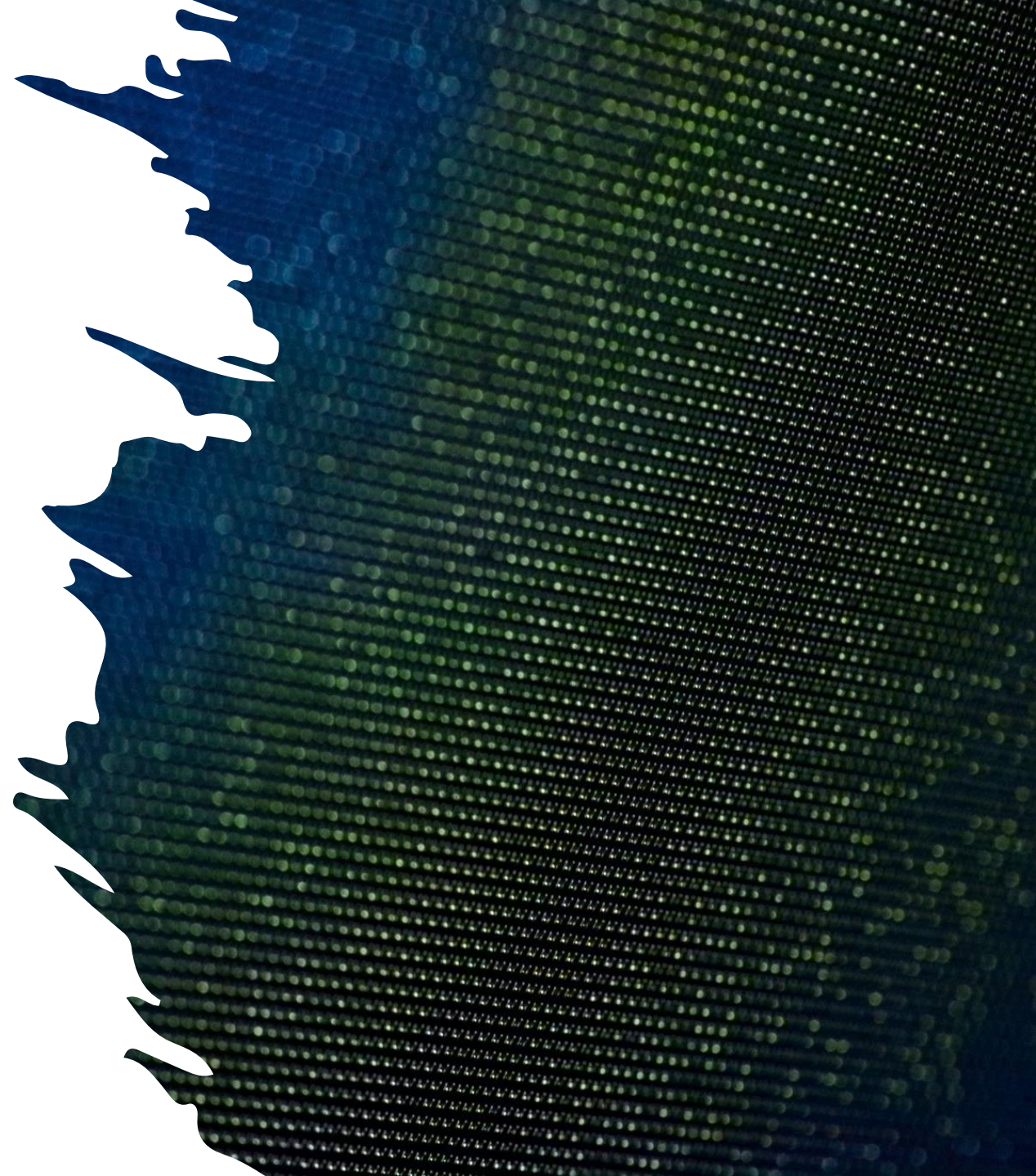


Estructuras de Datos

M.Sc Steven Santillan Padilla.



Agenda

- Tipos de Datos Genéricos
 - Por qué
 - Qué
 - Cómo (en Java)

Introducción

Motivación

- 4 métodos estáticos para dividir un número n:
 - `public static int divideByTwo (int n)`
 - `public static int divideByThree (int n)`
 - `public static int divideByFour (int n)`
 - `public static int divideByFive (int n)`

¿Cuál es el problema con esta estrategia?



Problemas

- Difícil de mantener
- No es escalable
- Es muy específica

¿Cuál es la solución?

Solución

- **Escribir un solo método**
 - Utilizar un parámetro para el divisor de la operación
 - `public static int divideBy (int n, int divisor)`
 - **¿Qué ganamos con esto?**
 - Ahora podemos enviar un argumento que defina el divisor
 - Logramos una solución **más genérica** (menos específica) y escalable
-

Parametrización de Datos

- Permite modificar el comportamiento de un código, sin necesidad de reescribirlo
- Es decir, permite la programación de rutinas **genéricas**
- Permite cambiar qué piezas de información enviamos a un método



Parametrización de Datos





int →

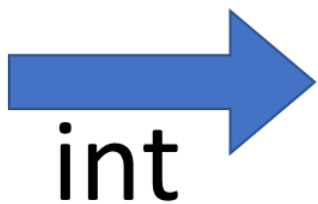
int →

divideBy

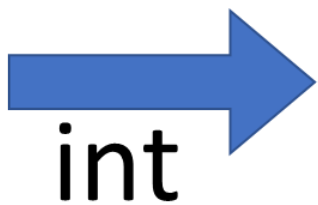
→ int



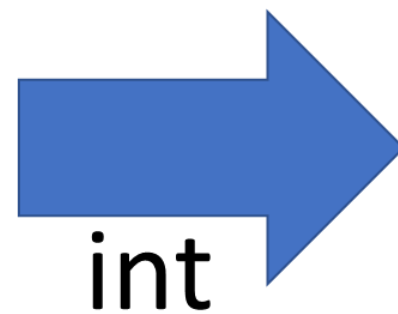
8



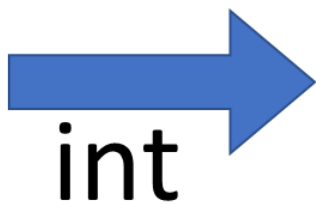
2



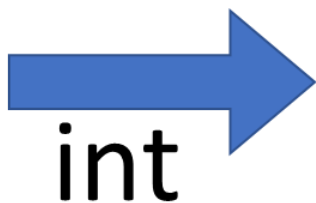
divideBy



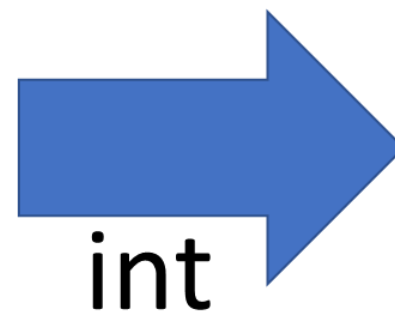
9



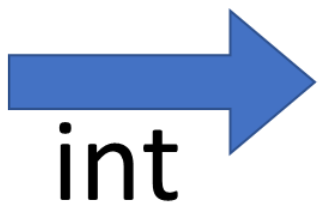
4



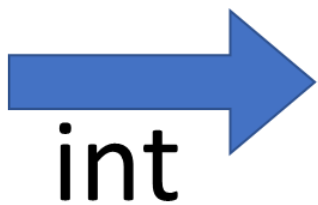
divideBy



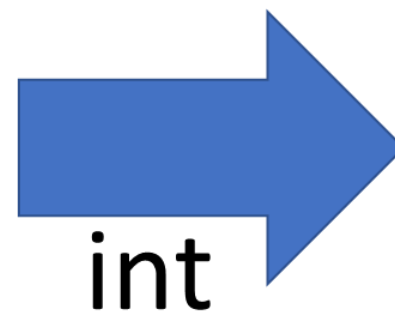
6



1



divideBy



Pero...

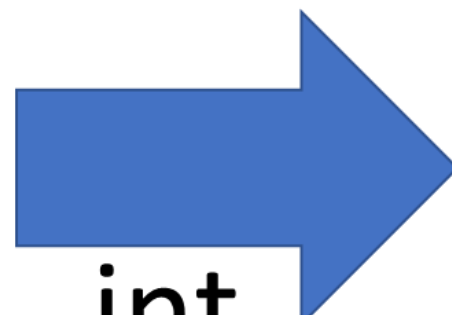


int

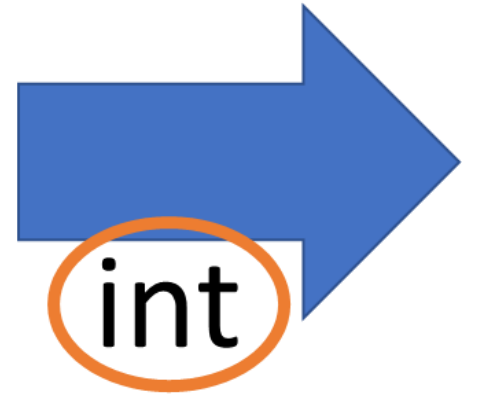
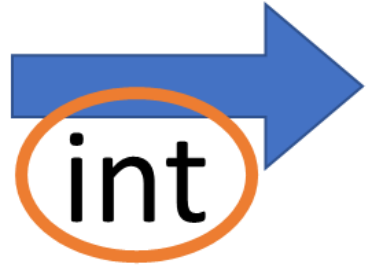
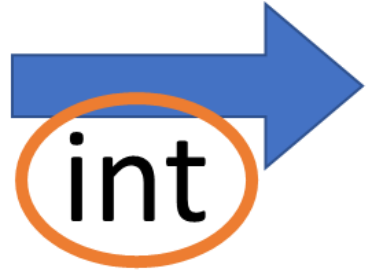


int

aMethod



int



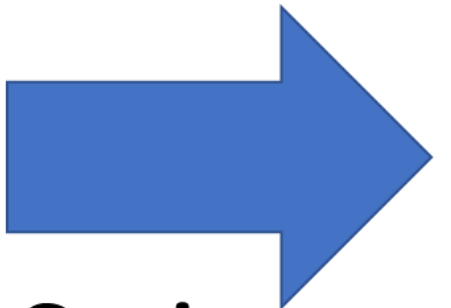


String



String

aMethod



String



aType →

aType →

aMethod

→ aType





¿Cómo lo
logramos?

Usando tipos de
datos genéricos





Tipos de Datos Genéricos



Parametrización de Tipos

- Permiten crear clases, interfaces y métodos en los que **los tipos de datos** sobre los que opera se especifican como parámetros.
- En una **parametrización de tipos** usamos parámetros de tipo.
- Así podemos crear, clases genéricas, interfaces genéricas, o métodos **genéricos**.

Ventajas

- El **código genérico** trabaja automáticamente con el tipo de datos pasados a su parámetro de tipo.
- Ejemplo: La búsqueda binaria es *la misma* ya sea que se esté buscando números, o cadenas de caracteres, o estudiantes, o materias, etc.
- Con genéricos, se puede definir el algoritmo una vez y aplicarlo a varios tipos de datos sin mayor esfuerzo adicional.

Genéricos en Java

Cómo se hace?

- Hay dos formas de usar tipos de datos genéricos en Java:

- Usando la clase Object



- Usando parámetros de tipo

La clase Object

Es la primera forma de usar genéricos en Java



Debido a que Object es la superclase de toda clase, una referencia de Object puede referirse a cualquier tipo de objeto.

Object aMethod(**Object** o)



aMethod(3)

aMethod("Hola")

aMethod(1.78)

aMethod(aCar)

Pero...



```
Object o = aMethod(new Student(...));
```

```
String theResult = (String) o;           int theResult = (int) o;
```

¿Cómo se hace?

Casts necesarios para convertir explícitamente de **Object** al tipo real de datos sobre los que se va a operar.



No es seguro: Posibles errores durante tiempo de ejecución cuando los casts no son posibles.

En 2004, Java introdujo *Generics* para solucionar este problema y lograr seguridad en tiempo de compilación.



Java Generics

Usted

Object

Escenario

Objetivo: Diseñar una clase que permita almacenar un artículo en una caja fuerte



article

A diagram showing a blue rectangular box representing a safe. Inside the box, the word 'article' is written in a large, black, sans-serif font.

Box

Una posible implementación

```
public class Box {  
  
    private String article = null;  
  
    public Box(String article) {  
        this.article = article;  
    }  
  
    public String getArticle() {  
        return article;  
    }  
  
    public void setArticle(String article) {  
        this.article = article;  
    }  
  
}
```

Una posible implementación

```
public class Box {  
    private String article = null;  
  
    public Box(String article) {  
        this.article = article;  
    }  
  
    public String getArticle() {  
        return article;  
    }  
  
    public void setArticle(String article) {  
        this.article = article;  
    }  
}
```

Una posible implementación

```
public class Box {  
  
    private String article = null;  
  
    public Box(String article) {  
        this.article = article;  
    }  
  
    public String getArticle() {  
        return article;  
    }  
  
    public void setArticle(String article) {  
        this.article = article;  
    }  
  
}
```

Uso

```
public class Main {  
    public static void main(String[] args) {  
        Box box1 = new Box("gold bar");  
        Box box2 = new Box("crown");  
        System.out.println(box1.getArticle());  
        System.out.println(box2.getArticle());  
    }  
}
```


Pero...

Solución limitada: artículos representados solo como objetos **String**

Requerimiento adicional: Se necesita almacenar mas información acerca de los artículos, permitiendo cualquier TDA.

Por ejemplo:

- TDA DiscoDuro

- TDA Joya

- TDA Laptop

- TDA Dinero

- TDA Camara

Usando Object

```
public class Box {  
  
    private Object article = null;  
  
    public Box(Object article) {  
        this.article = article;  
    }  
  
    public Object getArticle() {  
        return article;  
    }  
  
    public void setArticle(Object article) {  
        this.article = article;  
    }  
  
}
```



Usando Object

```
public class Box {  
    private Object article = null;  
  
    public Box(Object article) {  
        this.article = article;  
    }  
  
    public Object getArticle() {  
        return article;  
    }  
  
    public void setArticle(Object article) {  
        this.article = article;  
    }  
}
```



Usando Object

```
public class Box {  
  
    private Object article = null;  
  
    public Box(Object article) {  
        this.article = article;  
    }  
  
    public Object getArticle() {  
        return article;  
    }  
  
    public void setArticle(Object article) {  
        this.article = article;  
    }  
  
}
```



Usando Object



```
public class Main {  
    public static void main(String[] args) {  
        Box box1 = new Box("gold bar");  
        String article = box1.getArticle();  
    }  
}
```

Usando Object



```
public class Main {  
    public static void main(String[] args) {  
        Box box1 = new Box("gold bar");  
        String article = box1.getArticle();  
    }  
}
```

Usando Object



```
public class Main {  
    public static void main(String[] args) {  
        Box box1 = new Box("gold bar");  
        String article = (String) box1.getArticle();  
    }  
}
```

Usando Object



```
public class Main {  
    public static void main(String[] args) {  
        Box box1 = new Box("gold bar");  
        String article = (String) box1.getArticle();  
    }  
}
```


¿Cuál es el problema?

```
public class Box {  
    private Object article = null;  
  
    public Box(Object article) {  
        this.article = article;  
    }  
  
    public Object getArticle() {  
        return article;  
    }  
  
    public void setArticle(Object article) {  
        this.article = article;  
    }  
}
```



Generalizando

```
public class Box<T> {  
  
    private T article = null;  
  
    public Box(T article) {  
        this.article = article;  
    }  
  
    public T getArticle() {  
        return article;  
    }  
  
    public void setArticle(T article) {  
        this.article = article;  
    }  
  
}
```

<T>

```
public class Box<T> {  
  
    private T article = null;  
  
    public Box(T article) {  
        this.article = article;  
    }  
  
    public T getArticle() {  
        return article;  
    }  
  
    public void setArticle(T article) {  
        this.article = article;  
    }  
  
}
```

<T>

```
public class Box<T> {  
    private T article = null;  
  
    public Box(T article) {  
        this.article = article;  
    }  
  
    public T getArticle() {  
        return article;  
    }  
  
    public void setArticle(T article) {  
        this.article = article;  
    }  
}
```

T es un parámetro
de tipo

(siempre entre <>)

Box es una clase
genérica

Usando la clase genérica Box

```
public class Main {  
    public static void main(String[] args) {  
        Box<String> box1 = new Box("Joya");  
        Box<Integer> box2 = new Box(1000);  
        Box<Student> box3 = new Box(new Student ("Gonzalo", "Méndez"));  
        System.out.println(box1.getArticle());  
        System.out.println(box2.getArticle());  
        System.out.println(box3.getArticle());  
    }  
}
```



¿Qué son los Genéricos?

- **Definición:** Los genéricos en Java permiten escribir código que puede funcionar con cualquier tipo de dato.
 - **Motivación:**
 - Seguridad de tipo en tiempo de compilación.
 - Reutilización del código.
 - Sin necesidad de realizar casting.
-

Ejemplo Simple de Clase Genérica

- **T** es el parámetro de tipo que representa cualquier tipo de dato.
- Al crear una instancia de la clase, se define el tipo real.

//Ejemplo Simple de Clase Genérica

```
public class Caja<T> {  
    private T objeto;  
  
    public void set(T objeto) {  
        this.objeto = objeto;  
    }  
  
    public T get() {  
        return objeto;  
    }  
}
```

Ejemplo de Uso de Clase Genérica

- **Beneficios:**
 - Seguridad de tipo en tiempo de compilación.
 - Sin necesidad de casting al recuperar el valor.

```
public class Main {  
    public static void main(String[] args) {  
        // Caja que almacena String  
        Caja<String> cajaDeString = new Caja<>();  
        cajaDeString.set("Hola Mundo");  
        String mensaje = cajaDeString.get(); // Sin casting  
        System.out.println(mensaje);  
  
        // Caja que almacena Integer  
        Caja<Integer> cajaDeEntero = new Caja<>();  
        cajaDeEntero.set(123);  
        Integer numero = cajaDeEntero.get(); // Sin casting  
        System.out.println(numero);  
    }  
}
```


Genéricos vs. Object

- **Problemas:**

- Necesitas hacer casting cuando recuperas el valor.
- Riesgo de errores en tiempo de ejecución

//Sin Genéricos "Usando Object"

```
public class Caja {  
    private Object objeto;  
  
    public void set(Object objeto) {  
        this.objeto = objeto;  
    }  
  
    public Object get() {  
        return objeto;  
    }  
}
```

Genéricos vs. Object

- **Problemas:**

- Necesitas hacer casting cuando recuperas el valor.
- Riesgo de errores en tiempo de ejecución

//Sin Genéricos "Usando Object"

```
public class Caja {  
    private Object objeto;  
  
    public void set(Object objeto) {  
        this.objeto = objeto;  
    }  
  
    public Object get() {  
        return objeto;  
    }  
}
```

Característica	Con Object	Con Genéricos
Seguridad de Tipo	No, solo en tiempo de ejecución	Sí, en tiempo de compilación
Necesidad de Casting	Sí, siempre	No, el tipo es conocido
Riesgo de Errores	Alto (ClassCastException)	Bajo

**Genéricos
vs.
Object**

Métodos Genéricos

- Los métodos también pueden ser genéricos.
- `<T>` indica el tipo genérico que se pasa cuando llamas al método.
- El método imprimir puede manejar cualquier tipo de dato (String, Integer, Double).

//Métodos Genéricos

```
public class Utilidad {  
    public static <T> void imprimir(T objeto) {  
        System.out.println(objeto);  
    }  
}
```

// Ejemplo de Método Genérico

```
public class Main {  
    public static void main(String[] args) {  
        Utilidad.imprimir("Hola Mundo");  
        Utilidad.imprimir(123);  
        Utilidad.imprimir(99.99);  
    }  
}
```

Interfaces Genéricas

- Las interfaces también pueden ser genéricas.
- Permiten que las clases que las implementen trabajen con tipos parametrizados.

// Interfaces Genéricas

```
public interface Comparador<T> {  
    boolean comparar(T a, T b);  
}
```

// Ejemplo Interfaces Genéricas

```
public class ComparadorEnteros implements Comparador<Integer> {  
    public boolean comparar(Integer a, Integer b) {  
        return a > b;  
    }  
}
```

Comparador<Integer> compara dos enteros.

Uso de Wildcards (Comodines)

- **Comodines (?):** permiten mayor flexibilidad en el uso de tipos genéricos.
 - Tipos de comodines:
 - **Sin restricción:** ?
 - **Con límite superior:** ? extends T
 - **Con límite inferior:** ? super T
-

Ejemplo de Wildcard Sin Restricción

- **List<?>** acepta una lista de cualquier tipo de objeto.

```
//Ejemplo de Wildcard Sin Restricción
public class Utilidad {
    public static void imprimirLista(List<?> lista) {
        for (Object obj : lista) {
            System.out.println(obj);
        }
    }
}
```

Ejemplo de Wildcard con Límite Superior

- **List<? extends Number>** acepta listas de números (ej: Integer, Double, etc.).

```
//Ejemplo de Wildcard con Límite Superior
public class Utilidad {
    public static void imprimirNumeros(List<? extends Number>
lista) {
        for (Number numero : lista) {
            System.out.println(numero);
        }
    }
}
```




Resumen Genéricos

- **Genéricos en** permiten:
 - Reutilizar código para diferentes tipos de datos.
 - Mejorar la seguridad de tipo en tiempo de compilación.
 - Evitar el uso de casting innecesario.
 - **Aplicaciones:**
 - Clases, métodos e interfaces genéricos.
 - Uso en estructuras de datos como listas, pilas, colas.
-

Algunas Convenciones

- Los nombres de parámetros de tipo son:
 - Simples
 - Letras mayúsculas
- Nombres de tipos de parámetros más comúnmente usados
 - E - Element (usado ampliamente por el Framework de Colecciones Java)
 - K - Key
 - N - Number
 - T - Type
 - V - Value
 - S,U,V etc. - 2nd, 3rd, 4th types

E - Element (Elemento)

- Este parámetro es comúnmente utilizado en el contexto de colecciones que manejan elementos, como List, Set, etc. En el siguiente ejemplo, se utiliza E para representar un elemento de una lista genérica.

```
// Clase genérica que representa una caja de elementos
public class CajaElementos<E> {
    private E elemento;

    public void setElemento(E elemento) {
        this.elemento = elemento;
    }

    public E getElemento() {
        return elemento;
    }
}

public class Main {
    public static void main(String[] args) {
        // Caja que almacena un elemento de tipo String
        CajaElementos<String> cajaDeString = new
        CajaElementos<>();
        cajaDeString.setElemento("Elemento en la caja");
        System.out.println("Elemento: " +
        cajaDeString.getElemento());
    }
}
```

K - Key (Clave)

- Se usa principalmente en estructuras que tienen pares clave-valor, como un Map. Aquí, K representa la clave (key).

```
// Clase Par que representa un par clave-valor
public class Par<K, V> {
    private K clave;
    private V valor;

    public Par(K clave, V valor) {
        this.clave = clave;
        this.valor = valor;
    }

    public K getClave() {
        return clave;
    }

    public V getValor() {
        return valor;
    }
}

public class Main {
    public static void main(String[] args) {
        // Crear un par con una clave Integer y un valor String
        Par<Integer, String> par = new Par<>(1, "Valor asociado a la clave 1");
        System.out.println("Clave: " + par.getClave() + ", Valor: " + par.getValor());
    }
}
```

N - Number (Número)

- Este parámetro se utiliza cuando trabajas con clases o métodos genéricos que operan con tipos numéricos (como Integer, Double, etc.).

```
// Clase genérica que acepta solo números
public class Calculadora<N extends Number> {
    public double sumar(N numero1, N numero2) {
        return numero1.doubleValue() + numero2.doubleValue();
    }
}

public class Main {
    public static void main(String[] args) {
        // Calculadora que opera con números Integer
        Calculadora<Integer> calculadora = new Calculadora<>();
        System.out.println("Suma: " + calculadora.sumar(10, 20));

        // Calculadora que opera con números Double
        Calculadora<Double> calculadoraDouble = new
        Calculadora<>();
        System.out.println("Suma: " +
        calculadoraDouble.sumar(15.5, 4.5));
    }
}
```

T - Type (Tipo)

- Este es uno de los nombres más comunes para representar cualquier tipo genérico en clases o métodos.

```
// Clase genérica que acepta cualquier tipo de dato
public class Contenedor<T>{
    private T contenido;

    public void setContenido(T contenido){
        this.contenido = contenido;
    }

    public T getContenido(){
        return contenido;
    }
}

public class Main {
    public static void main(String[] args) {
        // Contenedor de tipo String
        Contenedor<String> contenedorDeString = new
Contenedor<>();
        contenedorDeString.setContenido("Texto genérico");
        System.out.println("Contenido: " +
contenedorDeString.getContenido());

        // Contenedor de tipo Integer
        Contenedor<Integer> contenedorDeInteger = new
Contenedor<>();
        contenedorDeInteger.setContenido(123);
        System.out.println("Contenido: " +
contenedorDeInteger.getContenido());
    }
}
```

V - Value (Valor)

- Este parámetro es comúnmente utilizado en estructuras que tienen pares clave-valor, como un Map. Aquí, V representa el valor asociado a la clave.

```
// Usando un Par genérico con clave K y valor V
public class ParClaveValor<K, V> {
    private K clave;
    private V valor;

    public ParClaveValor(K clave, V valor) {
        this.clave = clave;
        this.valor = valor;
    }

    public K getClave() {
        return clave;
    }

    public V getValor() {
        return valor;
    }
}

public class Main {
    public static void main(String[] args) {
        // Crear un Par con clave String y valor Integer
        ParClaveValor<String, Integer> par = new
        ParClaveValor<>("Clave1", 100);
        System.out.println("Clave: " + par.getClave() + ", Valor: " +
        par.getValor());
    }
}
```

S, U, V, etc. - 2nd, 3rd, 4th Types

- Estos parámetros se utilizan cuando necesitas manejar múltiples tipos en una clase o método genérico. Por ejemplo, T puede ser el primer tipo, y U o V pueden representar otros tipos adicionales.

```
public class Triple<T, U, V> {
    private T primerElemento;
    private U segundoElemento;
    private V tercerElemento;

    public Triple(T primerElemento, U segundoElemento, V
tercerElemento){
        this.primerElemento = primerElemento;
        this.segundoElemento = segundoElemento;
        this.tercerElemento = tercerElemento;
    }

    public T getPrimerElemento(){
        return primerElemento;
    }

    public U getSegundoElemento(){
        return segundoElemento;
    }

    public V getTercerElemento(){
        return tercerElemento;
    }
}

public class Main {
    public static void main(String[] args) {
        // Crear un Triple con diferentes tipos
        Triple<String, Integer, Double> triple = new
Triple<>("Texto", 100, 99.99);
        System.out.println("Primer Elemento: " +
triple.getPrimerElemento());
        System.out.println("Segundo Elemento: " +
triple.getSegundoElemento());
        System.out.println("Tercer Elemento: " +
triple.getTercerElemento());
    }
}
```




Resumen de las Convenciones:

- **E**: Elemento (usado en colecciones).
- **K**: Clave.
- **N**: Número.
- **T**: Tipo (Type).
- **V**: Valor.
- **S, U, V**: Representan segundos, terceros o cuartos tipos.

Cada convención tiene un uso específico para hacer que los nombres de los parámetros de tipo sean más descriptivos y fáciles de entender.

Otros Puntos Importantes

1. Los genéricos funcionan solo con tipos compuestos

```
Box<int> b1 = new Box(28); // ERROR
```

```
Box<char> b2 = new Box('a'); // ERROR
```

Pero sí se puede usar clases envoltorias:

```
Box<Integer> b1 = new Box(28); // OK
```

```
Box<Character> b2 = new Box('a'); // OK
```

2. Instancias de una clase generica pueden diferir en tipo

```
public class Main {  
    public static void main(String[] args) {  
        Box<String> box1 = new Box("Joya");  
        Box<Integer> box2 = new Box(1000);  
        box1 = box2;  
    }  
}
```

incompatible types: Box<Integer> cannot be converted to Box<String>

(Alt-Enter shows hints)

Box<String> != Box<Integer>

3. El parámetro de tipo no puede ser omitido

```
public class Main {  
    public static void main(String[] args) {  
        Box box3 = new Box("Joya");  
        String article2 = box3.getArticle();  
        Object article1 = box3.getArticle();  
        String article3 = (String) box3.getArticle();  
    }  
}
```

incompatible types: Object cannot be converted to String

(Alt-Enter shows hints)

4. Parametrización NO es overloading

Overloading permite a una clase tener más de un método con el mismo nombre, pero con una lista de argumentos diferentes.

```
static int add(int a, int b)  
static int add(String a, String b)
```

Parametrización permite *variar* el tipo de dato que un método recibirá

```
static <T> add(<T> a, <T> b)
```

Todas las versiones de un método overloaded retornan el mismo tipo de dato

Ejercicio 1:

Crear una clase que permita representar la **relación** entre dos entidades cualesquiera. A continuación, se muestran ejemplos de las relaciones que se quiere representar; la descripción de la relación entre las entidades aparece subrayada:

- Este cachorro es mascota de esta señora
 - Este empresario posee este conjunto de empresas
 - Esta ciudad es la capital de este país
 - Este veterinario atiende a este cachorro
 - Este avión se dirige a esta ciudad
-

```
public class Relation<T, R> {

    private T entitiy1;
    private R entitiy2;
    private String description;

    public Relation(T entitiy1, R entitiy2, String description) {
        this.entitiy1 = entitiy1;
        this.entitiy2 = entitiy2;
        this.description = description;
    }

    public T getEntitiy1() {
        return entitiy1;
    }

    public void setEntitiy1(T entitiy1) {
        this.entitiy1 = entitiy1;
    }

    public R getEntitiy2() {
        return entitiy2;
    }

    public void setEntitiy2(R entitiy2) {
        this.entitiy2 = entitiy2;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

}
```




Ejercicio 2:

- Crea una clase genérica llamada Par que almacene dos elementos.
 - Usa esta clase para almacenar pares de valores como `Par<String, Integer>`.
-



```
public class Par<K, V> {  
    private K clave;  
    private V valor;  
  
    public Par(K clave, V valor) {  
        this.clave = clave;  
        this.valor = valor;  
    }  
  
    public K getClave() {  
        return clave;  
    }  
  
    public V getValor() {  
        return valor;  
    }  
}
```

